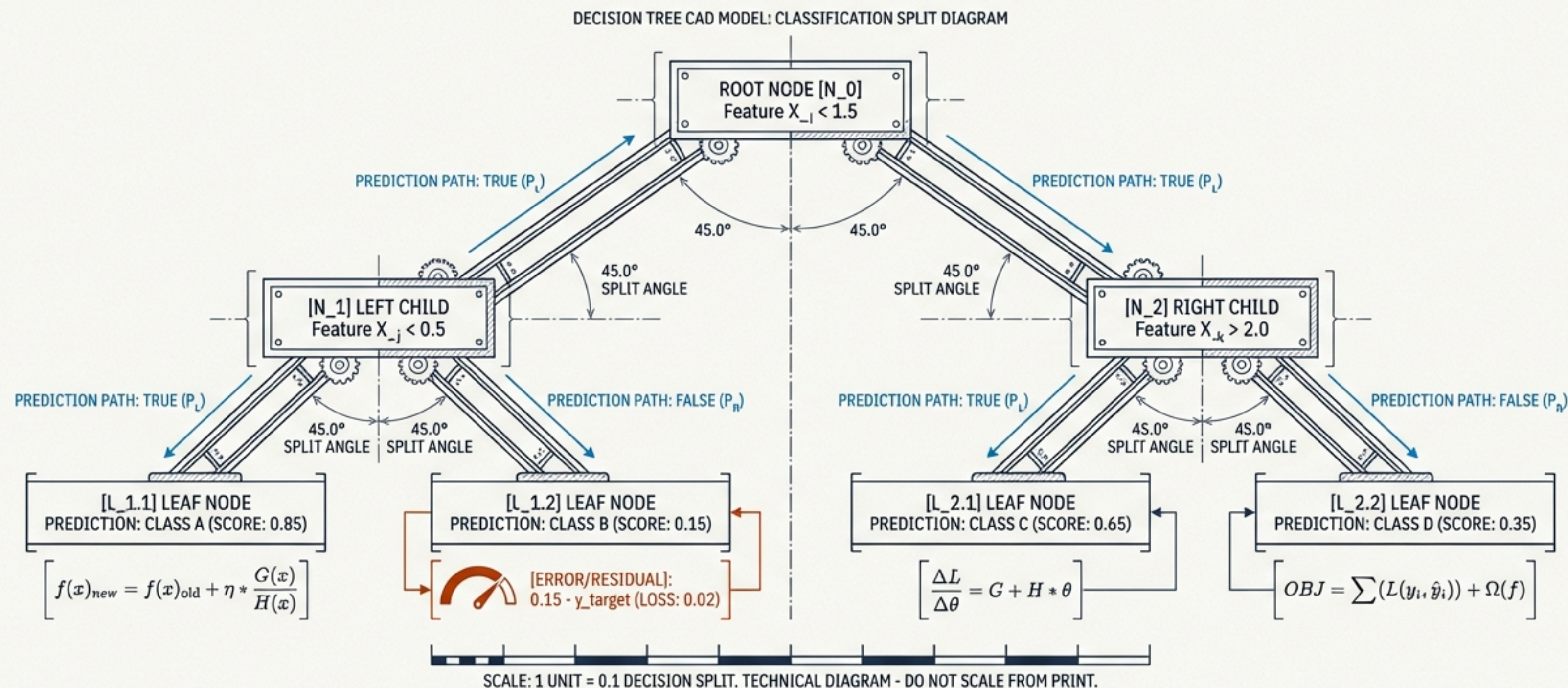


XGBoost for Classification: An Algorithm Autopsy

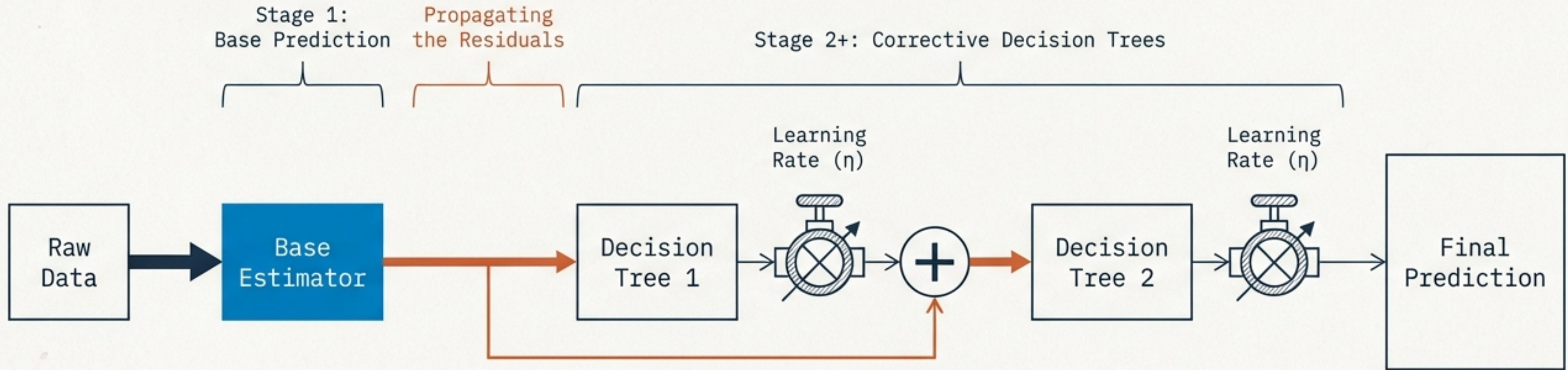
A manual, mathematical teardown of extreme gradient boosting—proving how complex models are just logical machines for crushing error.



> INIT_STAGE_WISE_ADDITIVE_MODELING...

The Assembly Line: Stage-Wise Additive Modelling

XGBoost does not solve the problem in one leap. It operates as a continuous assembly line. Each station builds a new model specifically engineered to fix the mistakes (residuals) made by the previous stations.



The Raw Material: A Toy Classification Problem

To observe the algorithm in action, we will track five students through the machine. Our goal: predict binary placement outcomes (0 = No, 1 = Yes) based solely on their CGPA.

CGPA (Feature)	Placement (Target)
5.70	0
6.24	0
7.10	1
8.15	1
9.60	1

Target Insight:
 $P(\text{Placement} = 1)$
is exactly $3/5$.

Architectural Upgrades: XGBoost vs. Vanilla Gradient Boosting

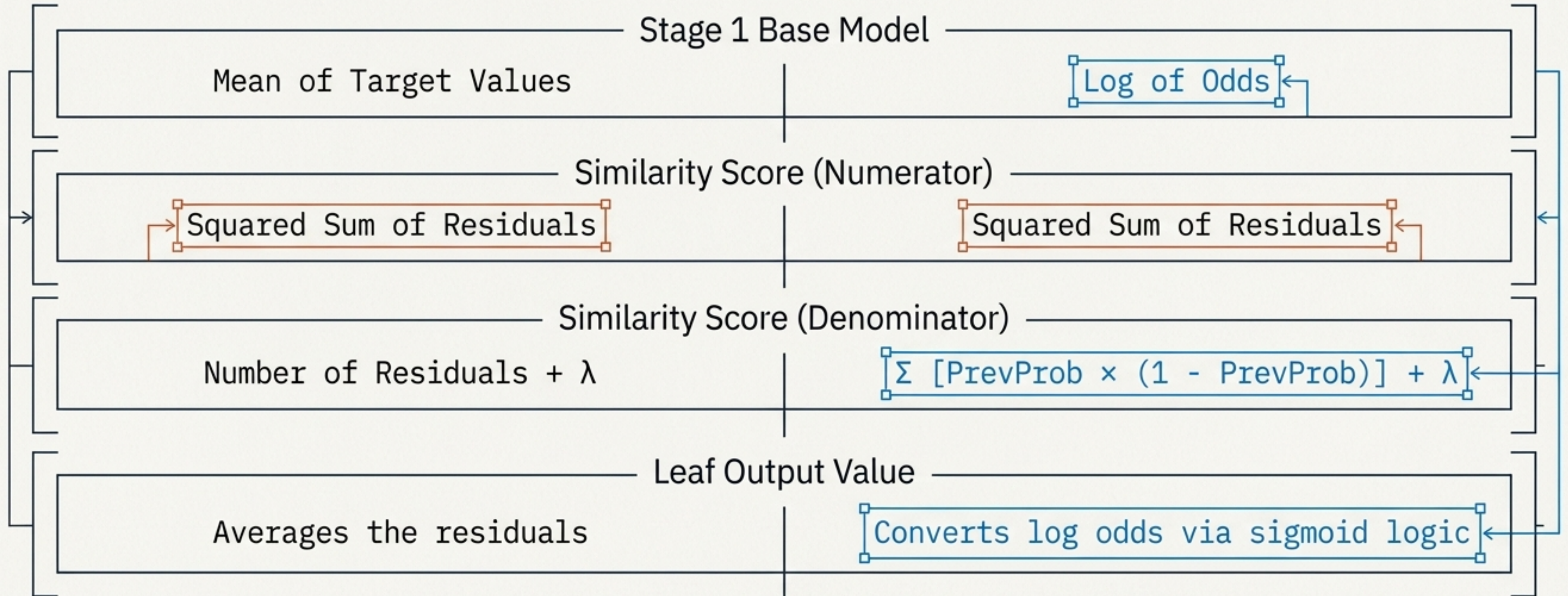
	Standard Gradient Boosting	XGBoost
Tree Architecture	Vanilla Decision Trees	Custom-built trees optimized for boosting
Splitting Metric	Gini Impurity / Entropy	Similarity Score (Mathematical purity)
Algorithmic Focus	Sequential error reduction	Extreme optimisation and regularisation

Takeaway: XGBoost isn't just a faster gradient booster; it **alters the fundamental mathematical metric** used to grow the trees.

System Diagnostic: Regression vs. Classification

Regression (Continuous Output)

Classification (Binary Output)



Stage 1: The Base Estimator (Log of Odds)

Unlike regression, binary classification cannot start with a simple mean. It requires a starting point mapped to probability space. We calculate the baseline Log of Odds for the entire dataset.

$$\text{Probability (p)} = \text{Positive Classes} / \text{Total Data}$$
$$p = 3 / 5 = 0.6$$

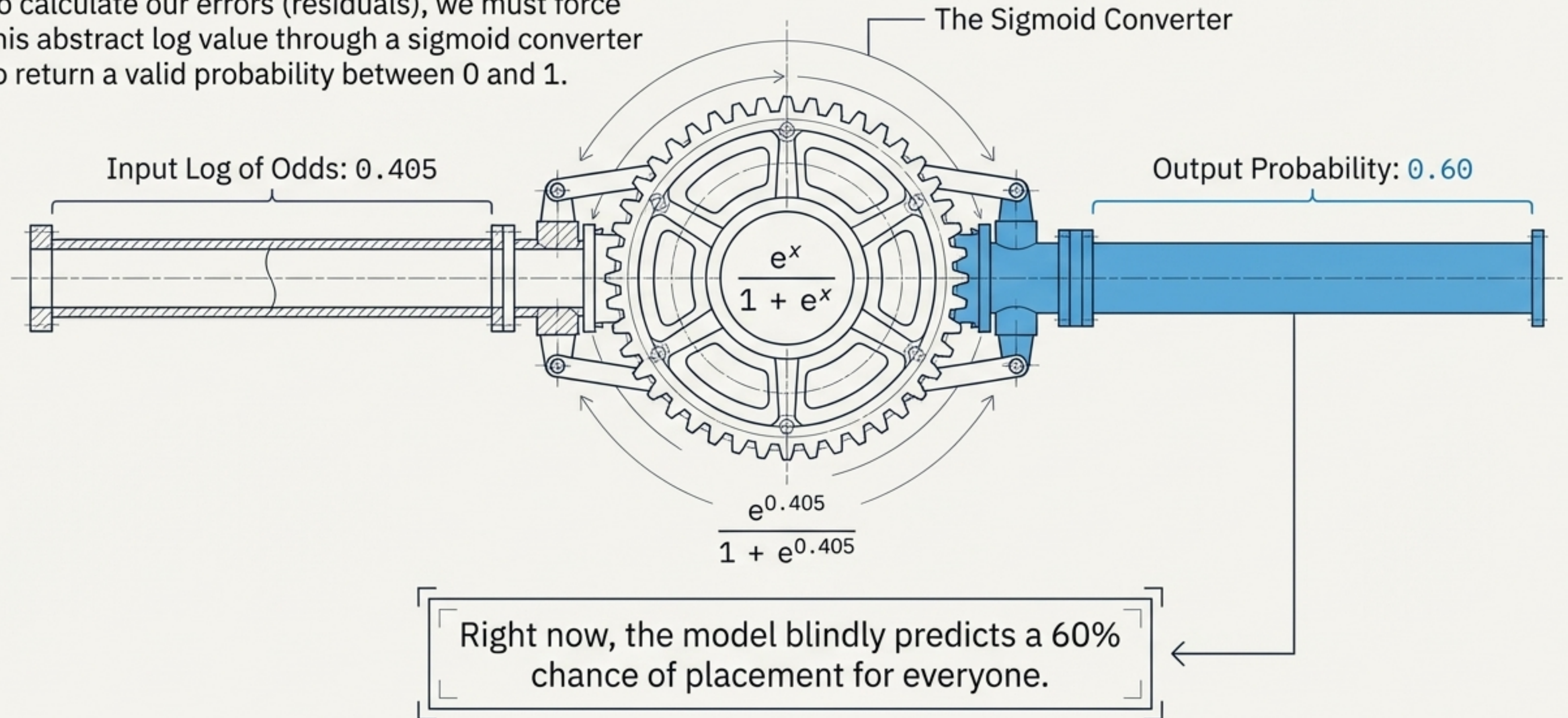
$$\text{Log of Odds} = \log(p / (1 - p))$$
$$\text{Log of Odds} = \log(0.6 / 0.4)$$
$$\text{Log of Odds} = \log(1.5)$$

$$\text{Final Base Output} = 0.405$$

Model 1 Output = 0.405 for ALL students, regardless of CGPA

The Translation Gear: Log Odds to Probability

"0.405" is a meaningless number in the real world. To calculate our errors (residuals), we must force this abstract log value through a sigmoid converter to return a valid probability between 0 and 1.



Generating the Protagonist: The Pseudo-Residuals

The algorithm now grades its own homework. It subtracts the base prediction from the actual reality. These resulting errors (residuals) become the sole target for the next stage of the machine.

True Target		Base Prob		Residual	
0	-	0.6	=	-0.6	
0	-	0.6	=	-0.6	
1	-	0.6	=		+0.4
1	-	0.6	=		+0.4
1	-	0.6	=		+0.4

Stage 2: The Anatomy of a Similarity Score

XGBoost grows custom decision trees. Instead of standard entropy, it uses "**Similarity Scores**" to group **similar residuals** together. A higher score means a purer, more similar cluster of errors.

$$\left(\sum \text{Residual}_i\right)^2$$

Squaring the sum of residuals rewards nodes that group errors of the same mathematical sign.

$$\sum [\text{PrevProb} \times (1 - \text{PrevProb})] + \lambda$$

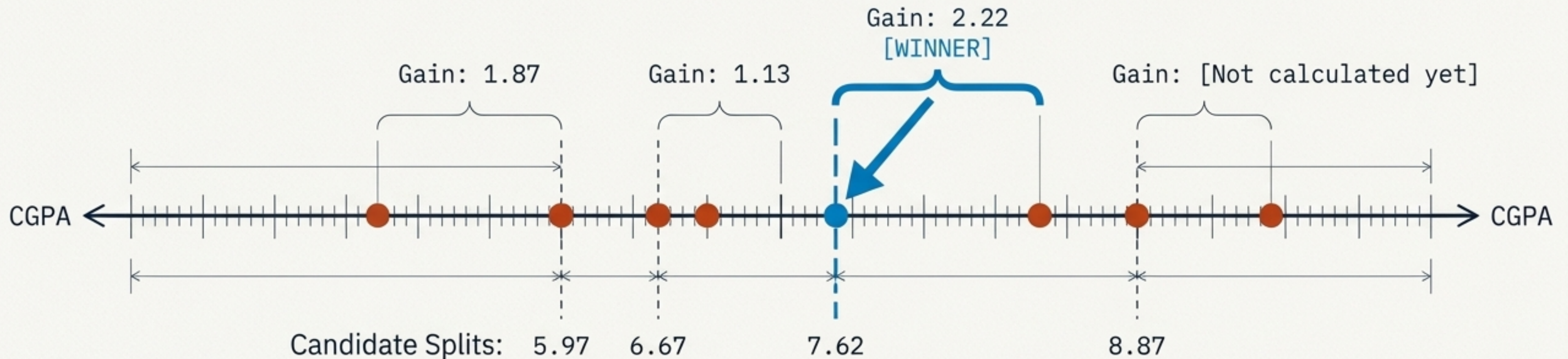
A variance scale factor based on the previous stage's probabilities.

The Regularisation Penalty. Set to 0 for this autopsy, but crucial for preventing overfitting in real models.

Root Node Similarity = 0 (because the sum of all Stage 1 residuals is exactly 0).

The Splitting Engine: Searching for Maximum Gain

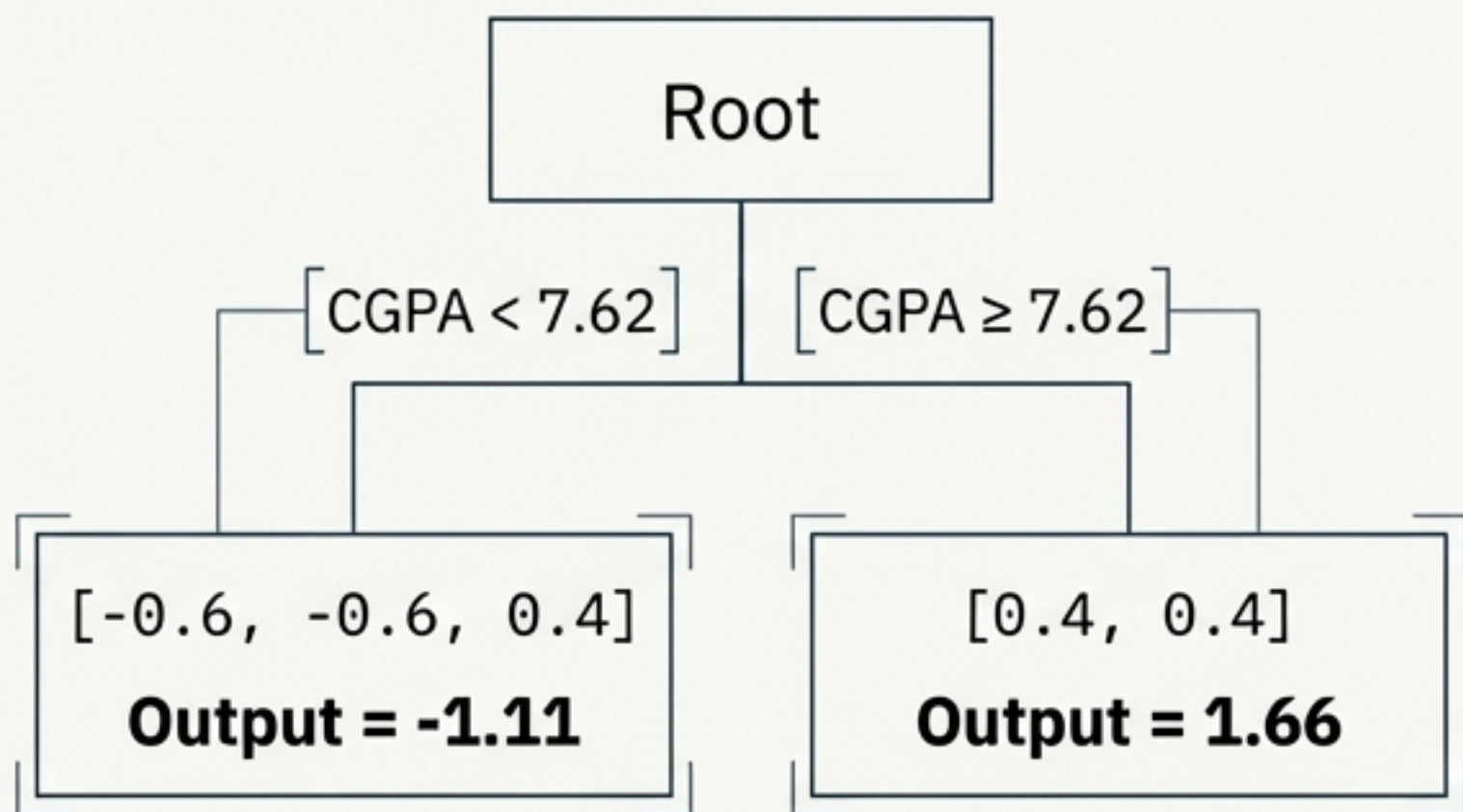
The algorithm treats the data like a continuous piece of **string**, testing every midpoint. It calculates the **Similarity Score** for the left piece, the right piece, and compares it to the root. It severs the data at the exact point that maximizes “Gain”.



$$\text{Gain} = \text{SS}(\text{Left Node}) + \text{SS}(\text{Right Node}) - \text{SS}(\text{Root})$$

Generating the Leaf Output Values

Once the optimal tree structure is locked, XGBoost calculates the exact numerical correction each leaf will output. The formula is nearly identical to the Similarity Score, but we remove the square from the numerator.



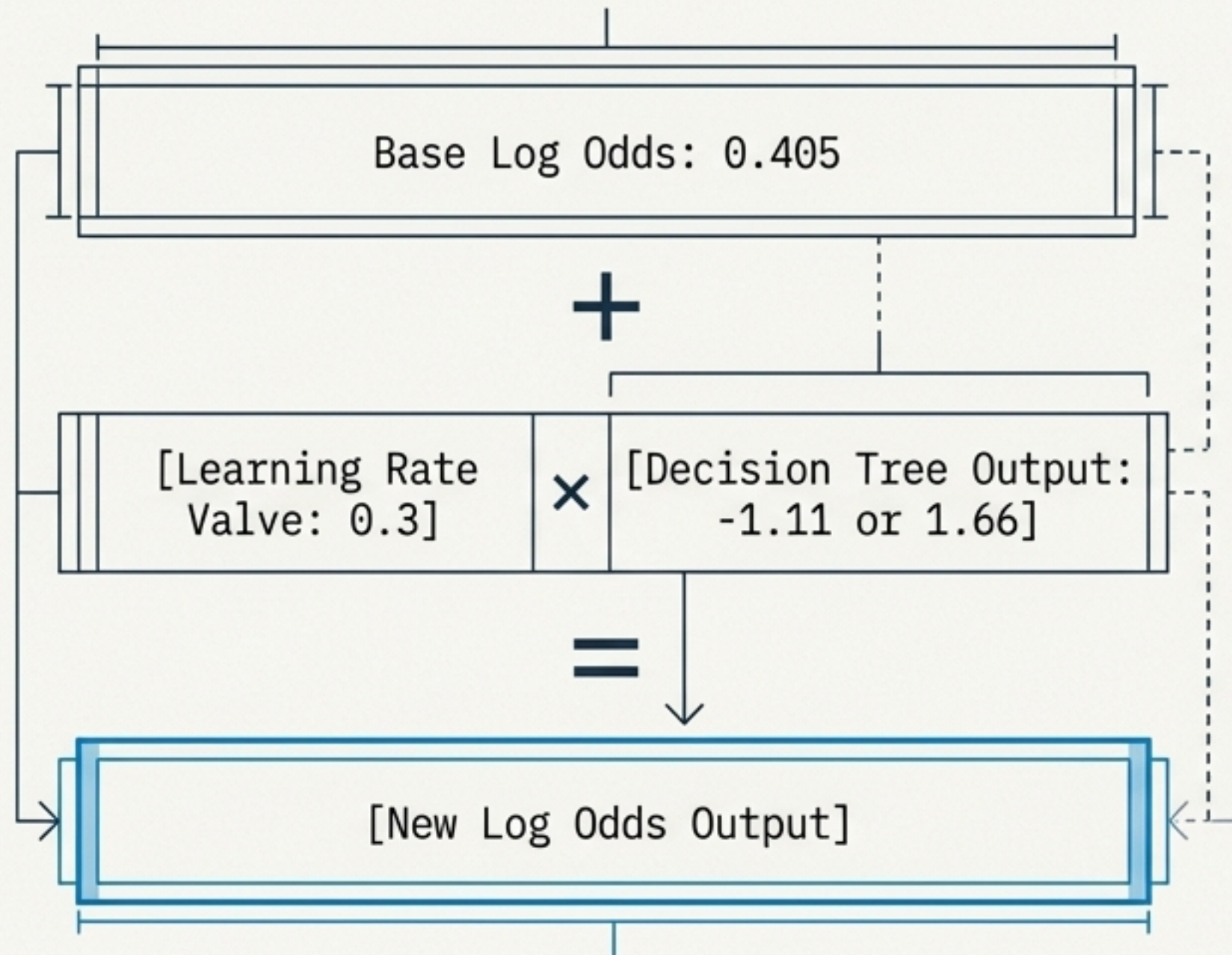
Output Value Formula

$$\frac{\sum \text{Residual}_i}{\sum [\text{PrevProb} \times (1 - \text{PrevProb})] + \lambda}$$

[No square '2' here]

Stage 2 Aggregation: Stacking the Models

The assembly line merges. To predict a student's placement, we take the Stage 1 baseline, add the Stage 2 tree correction, but throttle it with a Learning Rate (η) to ensure the model doesn't overcorrect.



Walkthrough Examples

Student 1 (CGPA 5.70):

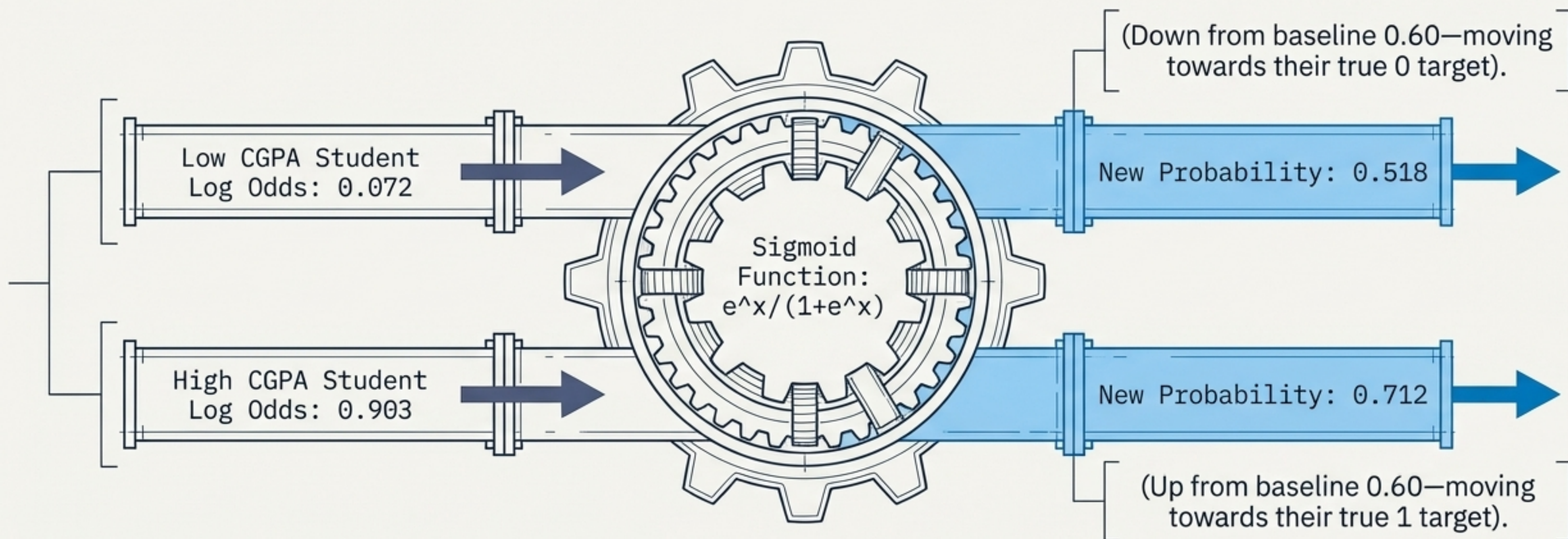
$$0.405 + (0.3 \times -1.11) = 0.072 \text{ (New Log Odds)}$$

Student 4 (CGPA 8.15):

$$0.405 + (0.3 \times 1.66) = 0.903 \text{ (New Log Odds)}$$

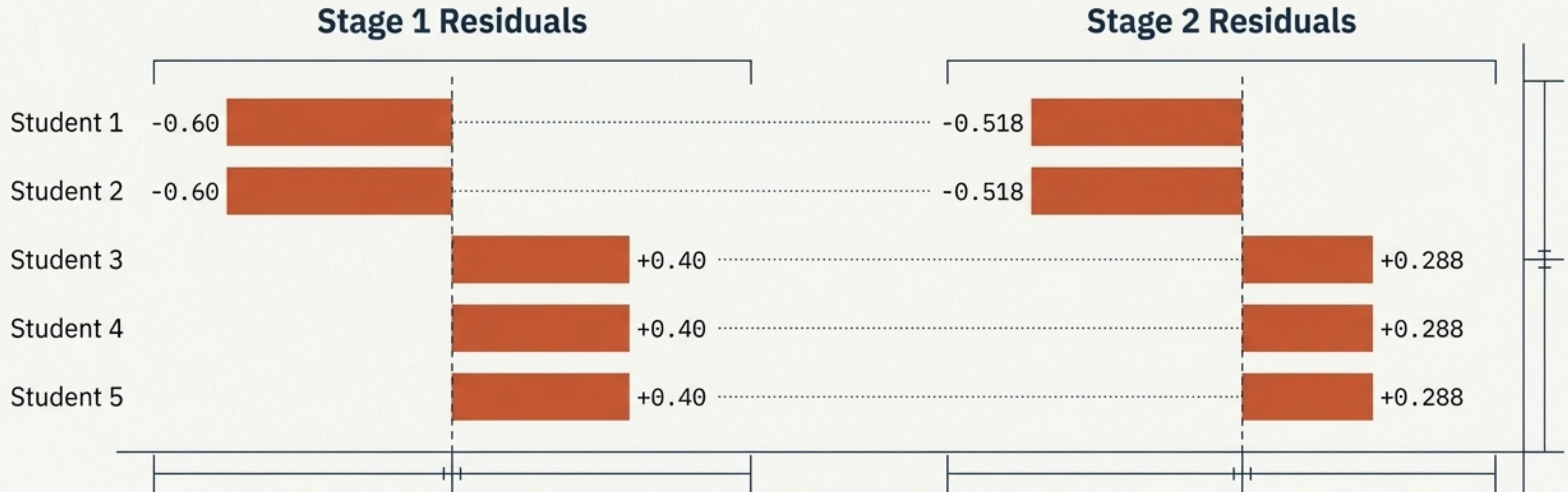
Re-Translation: Back to Probability Space

Our new aggregated predictions are still locked in Log Odds format. We push them back through the Sigmoid function to reveal the updated probabilities.



The Autopsy Results: Shrinking the Residuals

The mathematics worked. By training a specialized Similarity Tree on the initial mistakes and scaling it via the learning rate, the machine successfully pushed the prediction errors closer to zero across the board.

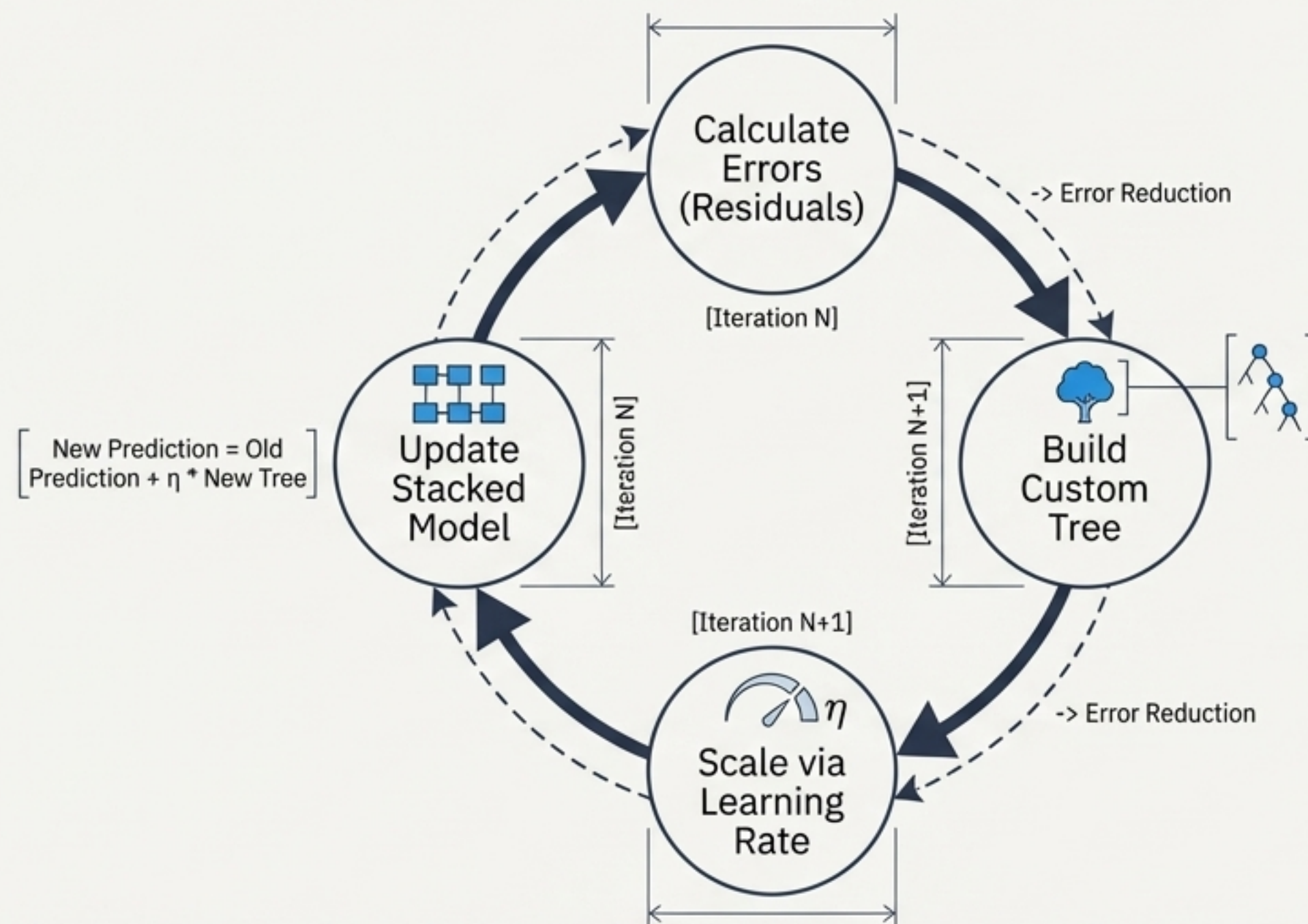


The protagonist—the Error—has been weakened.

The Infinite Assembly Line

We have manually completed one single stage of Extreme Gradient Boosting. In reality, the algorithm loops this exact process tens, hundreds, or thousands of times.

With every new tree, the residuals shrink further. The predictions become exponentially sharper. The assembly line continues until the error is crushed into insignificance.



> END_ROUTINE. Awaiting Mathematical Formulations in Stage 3...